

4.10 Code blocks and indentation

Settings

Beta features

code block

A code block is a series of statements grouped together.

Figure 4.10.1: Code blocks are indicated with indentation.

```
# First code block has no indentation

model = input("Enter car model: ")
year = int(input("Enter year of car manufacture: "))

antique = False
domestic = False

if year < 1970:
    # New code block has indentation of 4 columns
    antique = True

# Back to code block 0

if model in ["Ford", "Chevrolet", "Dodge"]:
    # New code block has indentation of 2 columns
    # Any amount of indentation > 0 is OK.
    domestic = True

# Back to code block 0

if antique:
    # New code block has indentation of 4 columns
    if domestic:
        # New block has 4 additional columns (8 total)
        print("My own model-T still runs like a charm...")
```

Enter car model:
Enter year of manufacture:
My own model-T still runs like a charm...

Try 4.10.1: Code blocks and indentation.

Fix the errors in the code below so that the program executes correctly.

Model Solution ▼

Figure 4.10.2: Some indentations are continuations of the previous line.

Multiple lines enclosed within parentheses are implicitly joined into a single string (without newlines) for very long strings or functions with numerous arguments. Ex: All extra lines are indented from the opening quotation mark on the first line.

When declaring list or dict literals, entries can be placed on separate lines for clarity.

```
declaration = ("When in the Course of human events, it becomes necessary for "
              "one people to dissolve the political bands which have connected "
              "them with another, and to assume among the powers of the earth."
              )

result_of_power = math.pow(long_variable_name_left_operand,
                           long_variable_name_right_operand)
```

Figure 4.10.3: List, dict multi-line constructs.

Containers like lists and dicts can be broken into multiple lines, with the elements on separate, indented lines.

```
my_list = [
    1, 2, 3,
    4, 5, 6
]

my_dict = {
    "entryA": 1,
    "entryB": 2
}
```

CHALLENGE ACTIVITY

4.10.1: Indentation: Fix the program.

763650.1722066.qx3zqy7

Drag and drop the lines of the following code in the same order, fixing the indentation as neces

```
if "New York" in temperatures:
    if temperatures["New York"] > 90:
        print("The city is melting!")
    else:
        print(f"The temperature in New York is {temperatures['N
else:
    print("The temperature in New York is unknown.")
```

[Click here for examples](#) ▼

How to use this tool ▼

Unused

```
print("The city is melting!")
print("The temperature in New York is unknown.")
if temperatures["New York"] > 90:
else:
print(f"The temperature in New York is {temperatures['N
if "New York" in temperatures:
else:
```

Check

View solution ▼ *(Instructors only)*

main.py

```
temperatures = {
    "Seattle": 56.5,
    "Kansas City": 81.9,
    "Los Angeles": 76.5
}

city = input()
temperature = float(input)
temperatures[city] = temp
```

**CHALLENGE
ACTIVITY**

4.10.2: Code blocks and indentation.

763650.1722066.qx3zqy7

Start

The following code contains at least one indentation error. Find and fix the error(s) so the program calculates the area of a triangle.

[Click here for example](#) ▼

```
1 # Fix the indentation in the code below
2     b = int(input())
3         h = int(input())
4             area = 0.5 * b * h
5
6 print(f"Base = {b} ft")
7 print(f"Height = {h} ft")
8 print(f"Area = {area:.3f} ft^2")
```

1

2

3

Check**Next level**[View solution](#) ▼ (Instructors only)